

CANDO: Conservative Action-Noise Directional Optimization for Residual Reinforcement Learning

Anonymous

Abstract—Pretrained generative policies provide strong behavior priors for robot control, but adapting them efficiently with online reinforcement learning remains difficult. A central challenge is how to improve a frozen generative policy without destroying the structure learned from offline demonstrations. Noise-space residual methods address this by modifying the policy’s input noise, but existing approaches are often sample-inefficient because fixed feasibility constraints are poorly aligned with value improvement. In contrast, action-space residual methods are more sample-efficient, but their efficiency can still be further improved. We propose Conservative Noise Directional Optimization (CNDO), a noise-space residual RL method that improves a frozen generative policy by learning state- and noise-conditioned directions around Gaussian latent samples. CNDO removes predefined latent feasible regions and instead regulates noise-space adaptation through soft conservative constraint, which consists of conservative value estimation and gated execution. We further introduce Conservative Action-Noise Directional Optimization (CANDO), which adds an action-space residual, combining structured latent-space improvement with local action correction. Experiments show that, once CNDO narrows the gap between noise-space and action-space residual learning, CANDO further improves over single-space optimization, achieving 76.1 NAUC and a 91.4% final success rate, surpassing the best single-space results by 6.6 NAUC and 4.1% success rate.

I. INTRODUCTION

Learning robot policies from offline demonstrations provides an effective way to acquire complex manipulation skills without manually designing task-specific controllers. In particular, generative policies based on diffusion models and conditional flow matching can represent complex and multimodal action distributions and have demonstrated strong performance across a broad range of robotic tasks [1], [2]. However, behavior cloning is inherently limited by the coverage of demonstration data and is susceptible to covariate shift during closed-loop deployment [3]. Online reinforcement learning (RL) provides a natural means to further improve pretrained policies through task feedback, but directly finetuning high-capacity generative policies typically requires optimization through the full generation process, which can introduce substantial computational cost and training instability [?], [4]. This motivates lightweight adaptation methods that keep the pretrained policy frozen while learning only a task-specific adaptation module.

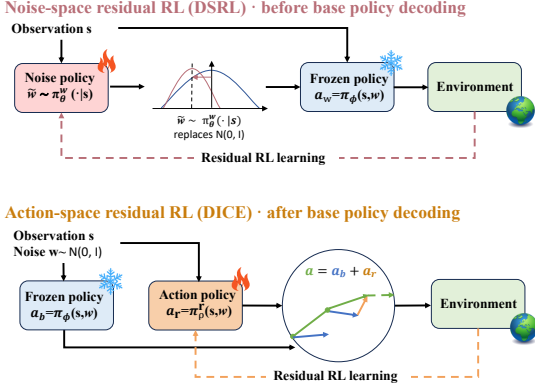
Residual reinforcement learning provides a natural framework for this setting by keeping a base policy fixed and learning only an additional residual correction [5]. For a frozen generative policy, residual adaptation can operate in two distinct spaces. In noise space, the input noise is modified before generation to alter the behavior produced by the

frozen policy; in action space, the generated action is directly corrected after decoding. Diffusion Steering via Reinforcement Learning (DSRL) represents the former approach by treating the input noise as the action of a latent-space Markov decision process [4]. In contrast, Distribution Contractive Reinforcement Learning (DICE) learns a lightweight residual over the generated action and combines residual optimization with value-guided action selection [6]. Although both spaces provide viable interfaces for online policy improvement, two important questions remain: can noise-space residual learning achieve sample efficiency comparable to strong action-space residual learning, and can the two spaces be jointly optimized to further improve policy learning?

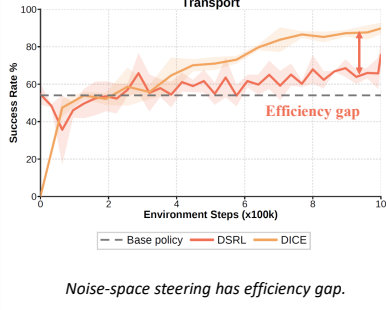
Existing noise-space methods are substantially less sample-efficient than DICE on most of the tasks considered in our experiments. We find that this gap is partly associated with how noise-space policy updates are regulated. DSRL restricts latent actions to a bounded region, Latent Policy Steering (LPS) projects them onto the typical Gaussian hypersphere, and Lagrangian Perturbation Diffusion Steering (LP-DS) retains Gaussian noise samples while constraining their residual perturbations through a Lagrangian trust region [4], [7], [8]. Such mechanisms improve training stability by explicitly limiting the region that the noise policy can explore. However, latent displacement is only indirectly related to the quality of the behavior produced after decoding. Consequently, a predefined geometric region may reject large but valuable latent edits while still admitting small edits that do not improve the resulting behavior. This suggests that noise-space adaptation should be regulated according to the quality of the decoded behavior, rather than solely by a predefined geometric region in latent space.

To address this gap, we propose **Conservative Noise Directional Optimization (CNDO)**, which removes the predefined latent feasible region and instead evaluates and regulates noise edits through their decoded behaviors. Removing hard geometric restrictions enables broader noise-space exploration, while increasing the risk of unstable optimization. CNDO introduces a soft conservative constraint consisting of two components: a conservative critic and gated execution. The conservative critic suppresses unsupported value overestimation when optimizing noise directions, while the gate selectively executes proposed edits according to target-critic value estimates and preserves the unmodified pretrained behavior as a fallback. In this way, CNDO expands the effective optimization range in noise space while preventing unrestricted exploration from destabilizing policy learning.

(a) Where residual RL intervene?



(b) Can noise-space residual learning achieve sample efficiency comparable to action-space residual learning



(c) Can both spaces be jointly optimized to further improve policy learning?

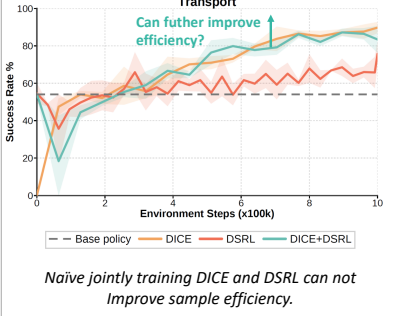


Fig. 1. Motivation for residual learning in noise and action spaces. (a) Noise-space residual RL edits the input noise before frozen-policy decoding, while action-space residual RL corrects decoded actions. (b) Existing noise-space methods show a clear sample-efficiency gap from DICE. (c) Naively combining DSRL and DICE provides no additional improvement.

We further combine CNDO with action-space residual learning and introduce **Conservative Action-Noise Directional Optimization (CANDO)**. The two residual branches affect the final policy through different pathways: the noise-space residual modifies the behavior generated through the frozen policy, whereas the action-space residual directly corrects the generated action. However, having access to both adaptation spaces does not necessarily guarantee additional gains from joint training. Our experiments show that the benefit of joint learning depends on the capabilities of both residual learners. When their standalone performance differs substantially, combining the two provides only limited improvement over the stronger learner. Once noise-space learning is improved to a level comparable to action-space residual learning, joint training can more effectively exploit policy improvements from both spaces.

Our main contributions can be summarized as follows:

- We identify a mismatch between hard latent-geometric constraints and decoded behavioral quality in noise-space residual learning, and propose **CNDO** to regulate noise-space adaptation through soft conservative constraint rather than a fixed latent feasible region.
- We further introduce **CANDO**, which jointly optimizes CNDO and action-space residual learning and achieves higher overall sample efficiency than the best single-space method.
- Through experiments, we reveal the sample-efficiency gap between noise-space and action-space residual learning and further show that joint training provides limited additional benefit when the standalone capabilities of the two residual learners are substantially mismatched.

II. RELATED WORK

A. Residual Reinforcement Learning

Residual reinforcement learning improves an existing controller or policy by preserving its effective behavioral prior and learning only the corrections that are missing from the base policy. Early works on Residual Policy Learning

and Residual Reinforcement Learning for Robot Control add trainable residual actions on top of hand-designed or model-based controllers, allowing reinforcement learning to avoid searching over the entire control policy from scratch [5], [9]. This paradigm was later extended to contact-rich manipulation with visual observations [10], shared autonomy [11], and settings where the base controller is obtained from demonstrations or motion primitives rather than manually designed [12], [13].

B. Noise-Space Policy Steering

Using latent representations to structure policy optimization predates generative robot policies. PLAS optimizes policies in a VAE-derived latent action space, while OPAL and SPiRL learn temporally extended latent primitives or skill priors from offline data [14]–[16]. LAPO further learns a latent-variable policy through advantage-weighted optimization to generate high-value actions [17]. With diffusion policies, the Gaussian noise input provides a native latent interface for modifying generated behavior. To the Noise and Back demonstrates that manipulating the diffusion process can effectively incorporate external guidance in shared autonomy [18]. More closely related to our setting, DSRL formulates the input noise of a frozen diffusion policy as the action of a latent-space MDP and optimizes it using reinforcement learning [4]. LPS directly optimizes a latent actor using gradients from an original-action-space critic, backpropagated through a differentiable one-step MeanFlow policy, and constrains the actor output to the Gaussian hypersphere [7]. More recently, LP-DS retains samples from the Gaussian prior and learns residual noise perturbations while regulating their magnitude through a Lagrangian trust-region constraint [8].

C. Action-Space Residual Reinforcement Learning

In contrast to noise-space steering before policy decoding, action-space residual learning directly corrects the output action. Recent work has extended the classical residual formulation to high-capacity policies pretrained from offline demonstrations. ResiP learns closed-loop per-step residual

corrections over frozen action-chunking policies to compensate for execution errors in precise assembly tasks [19]. Policy Decorator introduces a model-agnostic residual adapter for online refinement of large pretrained policies [20], while ResFiT applies off-policy RL to learn lightweight per-step corrections over behavior-cloning policies [21]. Related approaches also use value functions to coordinate pretrained behaviors with online policy improvement. IBRL treats actions proposed by an imitation policy as candidates for both exploration and temporal-difference bootstrapping [22]. DAWN further studies critic cold-start and scale mismatch between base and residual actions as key bottlenecks for value learning in residual RL [23]. Most closely related to our action-space component, DICE combines residual action optimization with selective behavioral regularization and value-guided multi-candidate action selection, providing a strong sample-efficient action-space baseline [6].

III. METHOD

We develop our method in two variants. First, we introduce **Conservative Noise Directional Optimization (CNDO)** to improve noise-space reinforcement learning without imposing a fixed feasible region on the edited latent. We then augment CNDO with a post-decoding action residual to obtain **Conservative Action-Noise Directional Optimization (CANDO)**, jointly exploiting policy improvement in noise and action spaces. Finally, we summarize the overall training and online interaction procedure.

A. Conservative Noise Directional Optimization

We consider an action-chunk Markov decision process. Given the current state or observation history s , a frozen Flow Matching policy generates a base action chunk a_b from Gaussian noise:

$$w \sim \mathcal{N}(0, I), \quad a_b = \pi_\phi(s, w), \quad (1)$$

where π_ϕ remains frozen throughout online training. We learn an action-value critic $Q_\omega(s, a)$ directly over decoded action chunks, such that noise-space policy improvements are ultimately evaluated by their induced behaviors.

Existing noise-space methods are shown in Fig. 2, these constraints stabilize policy optimization by explicitly restricting latent exploration. DSRL bounds each latent dimension

through a tanh actor, LPS normalizes latent variables onto the typical hypersphere of a high-dimensional Gaussian, and LP-DS combines residual noise perturbations with explicit bounds and a Lagrangian trust-region constraint [4], [7], [8]. Although these constraints improve stability, latent displacement is only an indirect indicator of decoded behavior quality. Large latent changes may produce valuable behaviors, while small changes do not necessarily imply better actions. Geometric constraints may therefore exclude useful noise-induced policy improvements and limit sample efficiency.

CNDO removes the fixed feasible-region constraint and instead learns a residual direction around each Gaussian anchor:

$$\begin{aligned} \delta w &= \pi_\theta^w(s, w), & \tilde{w} &= w + \delta w, \\ \tilde{a}_w &= \pi_\phi(s, \tilde{w}), & \delta a_w &= \tilde{a}_w - a_b. \end{aligned} \quad (2)$$

This Gaussian-anchored residual parameterization follows the same general form as LP-DS [8]. Instead, CNDO focuses on how such directions can be reliably optimized and executed after removing the explicit latent constraint.

Removing the hard constraint enables broader noise-space exploration, but also exposes the critic to decoded actions outside the available behavioral support. Critic extrapolation errors can assign spuriously high values to such actions and drive the direction actor toward increasingly unreliable latent regions. We address this issue with a **soft conservative constraint** consisting of two complementary components: **conservative critic** for reliable direction learning and **gated execution** for selective deployment.

For conservative critic, we maintain an ensemble of M critics and use their minimum whenever a conservative policy or target value is required. The critics are trained from expert and online replay using a multi-sample n -step TD objective:

$$\begin{aligned} \mathcal{L}_{TD} &= \sum_{j=1}^M \mathbb{E} \left[(Q_{\omega_j}(s_t, a_t) - y_t)^2 \right], \\ y_t &= R_t^{(n)} + \gamma^n (1 - d_t) \frac{1}{K_{\text{next}}} \sum_{k=1}^{K_{\text{next}}} Q_{\tilde{\omega}}(s_{t+n}, a'_k). \end{aligned} \quad (3)$$

The values of multiple next-policy samples are averaged to estimate the expected next-state value.

To further suppress extrapolation errors caused by unrestricted noise edits, we apply conservative regularization only

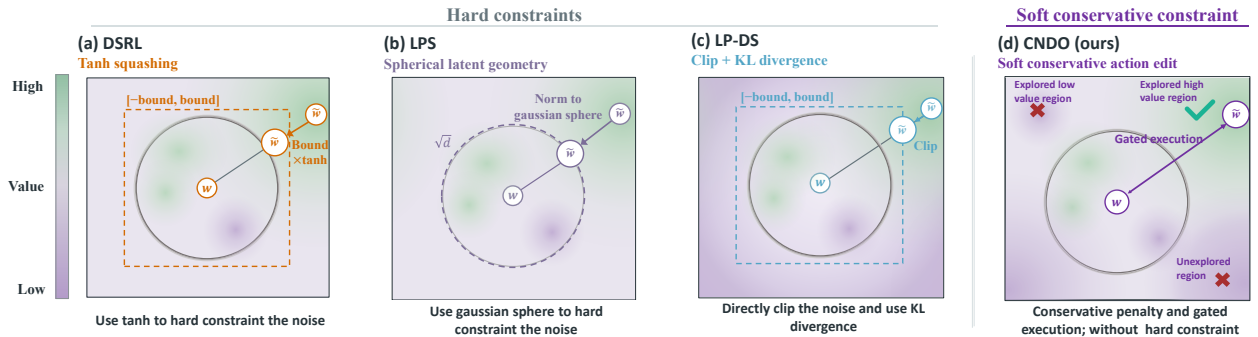


Fig. 2. Comparison of noise-space regulation strategies. (a) DSRL bounds latent variables with tanh squashing. (b) LPS projects latents onto the Gaussian hypersphere. (c) LP-DS constrains residual noise with clipping and a trust region. (d) CNDO removes fixed geometric constraints and instead uses conservative value estimation and gated execution.

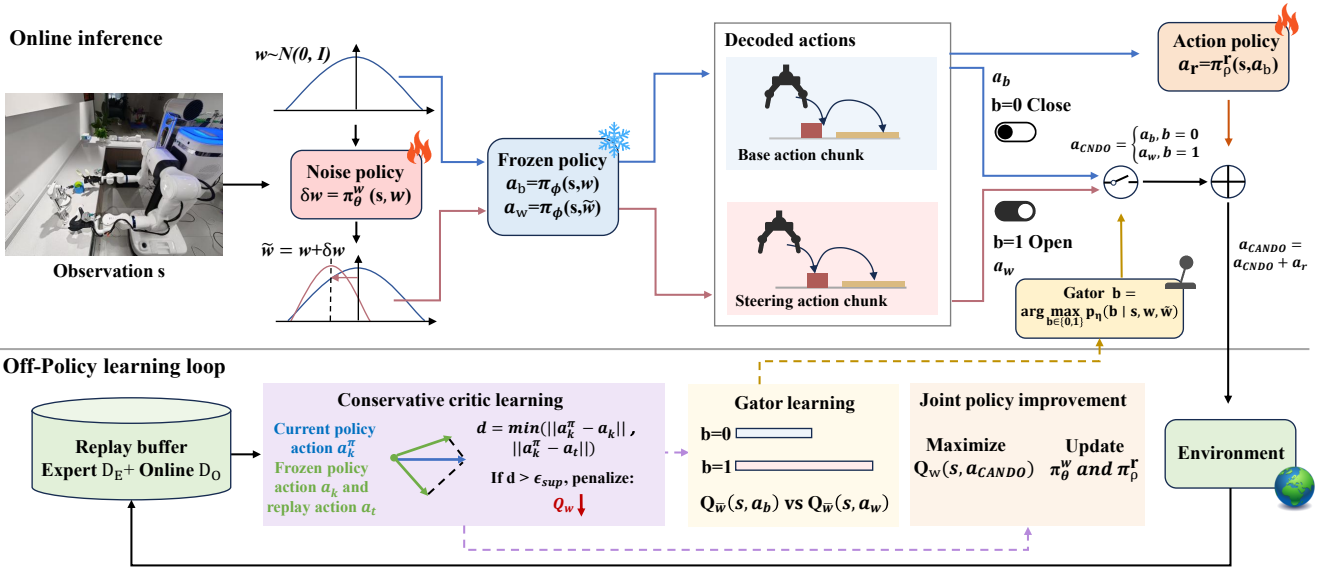


Fig. 3. Overview of CANDO. During online inference, a noise direction modifies the frozen-policy output, the gate selects whether to use the edit, and an action residual further refines the action. During off-policy learning, the critic is trained conservatively, the gate is supervised by the target critic, and both residual branches are jointly optimized. The pretrained policy remains frozen throughout.

to candidates outside behavioral support:

$$\begin{aligned} \mathcal{L}_Q &= \mathcal{L}_{TD} + \alpha_{CQL} \mathcal{L}_{CQL}, \\ \mathcal{L}_{CQL} &= \sum_{j=1}^M \frac{\sum_k m_k T \text{softplus} \left(\frac{Q_{\omega_j}(s_t, a_k^\pi) - Q_{\omega_j}(s_t, a_t) - m_Q}{T} \right)}{\max(\sum_k m_k, 1)}, \\ m_k &= \mathbb{I}[\min(\|a_k^\pi - a_k\|_2, \|a_k^\pi - a_t\|_2) > \epsilon_{\text{sup}}]. \end{aligned} \quad (4)$$

Here, a_k^π is a detached candidate generated by the current policy, while a_k is the frozen-policy action generated from the same Gaussian anchor. Thus, candidates supported by either the pretrained behavior or replay data remain unpenalized, while unsupported value overestimation is selectively suppressed.

The direction network directly optimizes the complete edited behavior:

$$\mathcal{L}_{\text{noise}} = -\mathbb{E}_{s,w} [Q_\omega(s, \tilde{a}_w)] + \beta_w \mathbb{E}_{s,w} [\|\delta w\|_2^2]. \quad (5)$$

Gradients are propagated through the frozen generative policy to π_θ^w , while the critic and pretrained-policy parameters remain fixed. The second term only provides a weak preference for smaller edits and does not impose a fixed latent feasible region.

Conservative critic reduces unreliable direction optimization, but critic estimates can still be inaccurate, especially early in training. CANDO therefore introduces a chunk-level binary gator G_η . Given $(s, w, \tilde{w})$, the gator predicts a distribution $p_\eta(b | s, w, \tilde{w})$ over $b \in \{0, 1\}$, and the deployed decision is

$$a_{\text{CANDO}} = \pi_{\text{CANDO}}(s, w) = a_b + b\delta a_w. \quad (6)$$

The gate is trained by distilling the relative values of these two behaviors from the target critic:

$$\mathcal{L}_{\text{gate}} = -\mathbb{E}_{s,w} \sum_{b \in \{0,1\}} q_{\tilde{w}}(b | s, w, \tilde{w}) \log p_\eta(b | s, w, \tilde{w}), \quad (7)$$

where $q_{\tilde{w}}$ is the teacher distribution obtained by normalizing and temperature-scaling the target-critic scores $Q_{\tilde{w}}(s, a_b + b\delta a_w) - \beta_\lambda b^2$. All quantities used to construct $q_{\tilde{w}}$ are detached from the direction network.

The direction network therefore uses the online critic to propose valuable edits, while the gate uses the more stable target critic to decide whether each edit should be executed. Together with conservative critic, this gated execution forms the soft conservative constraint of CANDO.

B. Action-Noise Joint Residual Training

CANDO adapts the frozen policy before decoding by editing its input noise. To additionally exploit the post-decoding action space, we introduce an action residual conditioned on the state and action. Specifically, CANDO is defined as

$$\begin{aligned} a_{\text{CANDO}} &= \pi_{\text{CANDO}}(s, w, a_b) \\ &= \pi_{\text{CANDO}}(s, w) + a_r \\ &= a_b + b\delta a_w + a_r, \\ a_r &= \pi_r^\rho(s, a_b), \end{aligned} \quad (8)$$

where a_r is residual action in action space.

The two branches are jointly optimized at the complete edited endpoint:

$$\begin{aligned} \mathcal{L}_{\text{joint}} &= -\mathbb{E}_{s,w} [Q_\omega(s, a_{\text{CANDO}})] \\ &\quad + \beta_w \mathbb{E}_{s,w} [\|\delta w\|_2^2] + \beta_r \mathbb{E}_{s,w} [\|a_r\|_2^2]. \end{aligned} \quad (9)$$

The first term jointly improves the noise direction and action residual, while the remaining terms weakly regularize their respective magnitudes. The action residual is applied after gating and therefore remains active when $b = 0$, in which case the executed action is $a_b + a_r$.

CANDO retains the same gate-learning procedure as CANDO. CANDO consequently combines two distinct policy-improvement paths: the noise branch modifies behavior through the frozen generator, whereas the action branch directly refines the decoded action.

Algorithm 1 Conservative Action-Noise Directional Optimization (CANDO)

```

1: Input: Expert replay  $\mathcal{D}_E$ , frozen policy  $\pi_\phi$ , update steps  $U$ , and interaction steps  $T$ 
2: Initialize: Online replay  $\mathcal{D}_O$ , direction network  $\pi_\theta^w$ , action-residual network  $\pi_\rho^r$ , gate network  $G_\eta$ , critics  $Q_\omega$ , and target critics  $Q_{\bar{\omega}}$ 
3: while not converged do
    //Off-policy parameter learning
4:   for update step  $u = 1$  to  $U$  do
5:     Sample a batch  $\mathcal{B} \sim \text{RLPD}(\mathcal{D}_E, \mathcal{D}_O)$ 
6:     Sample  $w \sim \mathcal{N}(0, I)$  and compute  $a_b = \pi_\phi(s, w)$ 
7:     Compute  $\delta w = \pi_\theta^w(s, w)$ ,  $\tilde{a}_w = \pi_\phi(s, w + \delta w)$ , and  $\delta a_w = \tilde{a}_w - a_b$ 
8:     Compute  $a_r = \pi_\rho^r(s, a_b)$ 
9:     Infer  $b = \arg \max_{b \in \{0,1\}} p_\eta(b | s, w, \tilde{w})$ 
10:    Construct the deployable CANDO action  $a = a_b + b \delta a_w + a_r$ 
11:    Generate next-policy actions and conservative candidates using the complete CANDO policy
12:    Update critics  $\omega$  using Eq. (4)
13:    Update gator  $\eta$  using Eq. (7)
14:    Update actors  $\theta$  and  $\rho$  using Eq. (9)
15:    Update target critics by Polyak averaging  $\bar{\omega} = (1 - \tau)\bar{\omega} + \tau\omega$ 
16:  end for
    //Online data collection
17:  for interaction step  $t = 1$  to  $T$  do
18:    Get current observation  $s_t$  and sample  $w_t \sim \mathcal{N}(0, I)$ 
19:    Compute  $a_{b,t} = \pi_\phi(s_t, w_t)$ ,  $\delta w_t = \pi_\theta^w(s_t, w_t)$ , and  $\tilde{a}_{w,t} = \pi_\phi(s_t, w_t + \delta w_t)$ 
20:    Compute  $\delta a_{w,t} = \tilde{a}_{w,t} - a_{b,t}$  and  $a_{r,t} = \pi_\rho^r(s_t, a_{b,t})$ 
21:    Infer  $b_t = \arg \max_{b \in \{0,1\}} p_\eta(b_t | s_t, w_t, w_t + \delta w_t)$ 
22:    Execute  $a_t = a_{b,t} + b_t \delta a_{w,t} + a_{r,t}$ 
23:    Add  $(s_t, a_t, r_t, s_{t+1})$  to  $\mathcal{D}_O$ 
24:  end for
25: end while

```

C. Overall CANDO Framework

We briefly introduce the CANDO in Alg. 1. CANDO alternates between off-policy parameter updates and online data collection. During each update, we use RLDPD [24] sample states and transitions from a mixture of expert and online replay, and use newly sampled Gaussian latents to construct the original behavior a , the noise-edited behavior $\tilde{a}_w$, and the action residual a_r . The complete CANDO policy is used to generate the next-policy actions and conservative candidates for critic learning. We first update the critic using Eq. (4). Then, the gator is distilled from the target critic using Eq. (7), and the target critic is updated by Polyak averaging. The pretrained Flow Matching policy remains frozen throughout this process. The jointly optimize the noise-direction network and action-residual network using Eq. (9).

IV. EXPERIMENTS

We design the experiments to answer the following questions:

- **Q1:** Can CNDO improve noise-space reinforcement learning to a level comparable to strong action-space residual methods?
- **Q2:** How do geometric latent constraints, conservative value learning, and gated execution affect noise-space policy improvement?
- **Q3:** How does the standalone capability of noise-space and action-space learners affect the benefit of joint residual training?

A. Experimental Setup

Tasks and data. We use state observations on five manipulation tasks from two benchmarks: *Can*, *Square*, and *Transport* from RoboMimic [25], and *PegInsertionSide* and *StackCube* from ManiSkill [26]. The frozen Flow Matching policy is pretrained on the corresponding task’s separate pretraining data.

Online training and evaluation. Within each task, every method uses the same frozen base-policy checkpoint, expert data, action horizon, and normalization. Training uses 16 parallel environments, batch size 256, ten gradient steps per collection iteration, three-step returns, and discount $\gamma = 0.99$. We run three seeds 42, 43, and 44 and report the mean and sample standard deviation. Success is the binary completion signal defined by each environment.

Metrics. Let B be the max environment steps of each task, and let $\text{SR}(n) \in [0, 1]$ denote success rate after n environment transitions. We measure sample efficiency by

$$\text{NAUC} = \frac{100}{B} \int_0^B \text{SR}(n) \, \text{d}n, \quad (10)$$

using linear interpolation between evaluation points and trapezoidal integration. We also report $100 \text{SR}(B)$ as **Final SR**. The overall score gives every task equal weight. The frozen behavior-policy result is treated as a constant curve.

Baseline algorithms and proposed method. We compare the following frozen-policy adaptation methods:

- 1) *Base policy*: the pretrained frozen Flow Matching behavior policy without online adaptation.
- 2) *DSRL* [4]: a noise-space method that learns latent inputs for the frozen generative policy.
- 3) *LPS* [7]: a latent-policy method that directly propagates action-space value gradients through a differentiable flow decoder while constraining the latent input to the typical Gaussian hypersphere.
- 4) *LP-DS* [8]: residual noise steering with an explicit latent bound and a Lagrangian trust-region constraint.
- 5) *DICE* [6]: an action-space residual learner with value-guided candidate selection.
- 6) *DSRL+DICE*: the direct, simultaneous combination of DSRL noise learning and DICE action residual learning.
- 7) *CNDO*: our noise-space directional learner with conservative value estimation and gated execution.
- 8) *CANDO*: our joint CNDO and action-residual learner.

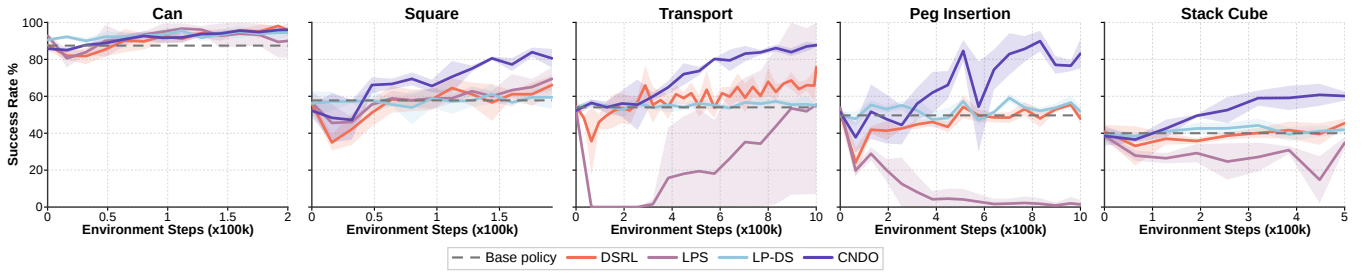


Fig. 4. Noise-space policy improvement on five tasks. Lines and shaded regions show the mean and sample standard deviation over three seeds; the gray dashed line is the frozen base policy.

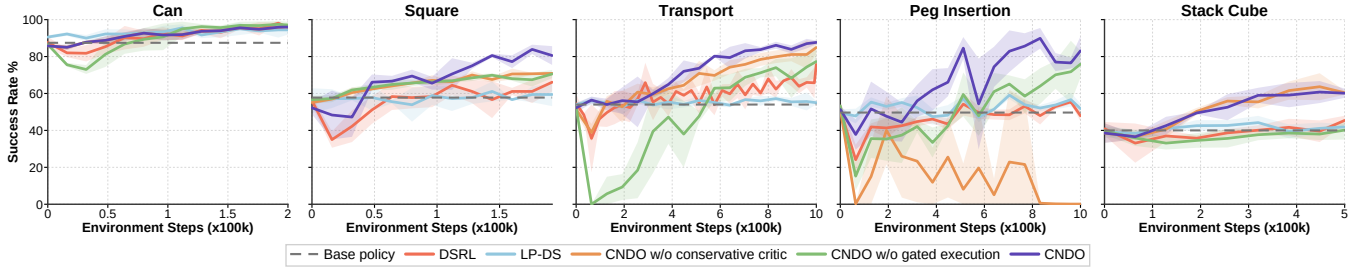


Fig. 5. CNDO component analysis on five tasks. CNDO “w/o gated execution” always executes the full latent edit, and “w/o conservative critic” sets the behavior-support conservative loss weight to zero. Lines and shaded regions denote the mean and sample standard deviation over three seeds; the gray dashed line is the frozen base policy.

B. Noise-Space Policy Improvement

This experiment addresses **Q1**. We compare CNDO with existing noise-space methods and use DICE as the action-space reference. Fig. 4 and Tab. I show that CNDO consistently improves over the frozen policy and is the strongest noise-space method on average. Relative to DSRL, CNDO raises average NAUC from 57.9 to 69.5 and Final SR from 66.2% to 81.5%; relative to LP-DS, the gains are 9.6 NAUC points and 21.0 Final-SR points. Relative to the common-backbone LPS port, the gains are 27.8 NAUC points and 31.3 Final-SR points. The improvement is particularly large on Transport, PegInsertionSide, and StackCube. Can is the exception in NAUC: its pretrained success rate is already 87.4%, and LP-DS reaches 93.0 NAUC compared with 91.6 for CNDO and 91.4 for LPS.

The LPS port obtains 41.7 average NAUC and 50.2% average Final SR. It improves over the frozen endpoint on Can and Square, reaching 90.0% and 69.4% Final SR, respectively, and approximately preserves the Transport endpoint on average. However, its learning is highly variable on Transport: the three final success rates are 0.0%, 77.3%, and 89.0%. On PegInsertionSide, all three runs deteriorate and finish at 0.0%, 0.0%, and 4.3%, while on StackCube the mean Final SR is 34.8%, below the 40.0% frozen policy. Thus, direct action-value gradients and typical-shell projection alone do not yield robust online improvement when transferred to our multi-step Flow backbone. This result should not be interpreted as an exact evaluation of the original one-step MeanFlow implementation; it specifically exposes the sensitivity of its steering mechanism under the shared-backbone protocol.

CNDO also closes the *sample-efficiency* gap to the action-space reference: its average NAUC is 69.5, compared with

68.5 for DICE. The task-level comparison is more nuanced. CNDO has higher NAUC on Square, Transport, and PegInsertionSide, while DICE is better on Can and substantially better on StackCube. DICE also has the higher average Final SR (87.3% versus 81.5%), mostly because it reaches 94.1% on StackCube. Thus, CNDO makes noise-space learning competitive with strong action residuals in overall sample efficiency without dominating DICE on every task.

C. Ablation Studies

This experiment addresses **Q2**. We compare full CNDO with different components that set the conservative critic weight to zero or force the execution gate to one. In the component curve shown in Fig. 5, LP-DS supplies the geometric-constraint comparator also with directional optimization: unlike CNDO, it enforces noise with a fixed latent bound.

Removing conservative value regularization lowers average NAUC from 69.5 to 58.2. Its effect is task dependent, but it prevents a severe failure on PegInsertionSide, where Final SR falls from 83.1% to 0.1% without it. This supports using behavior-support-aware conservatism when the fixed latent feasible region is removed.

Always executing the edit lowers average NAUC to 57.5 and Final SR to 72.2%. The largest NAUC losses occur on Transport (71.2 to 46.4), PegInsertionSide (65.8 to 49.7), and StackCube (50.9 to 36.6). The gate primarily improves sample efficiency and cross-task robustness, rather than guaranteeing a higher endpoint on every task. Finally, full CNDO exceeds the geometrically constrained LP-DS comparator by 9.6 average NAUC points, consistent with the hypothesis that a fixed latent region can reject useful decoded improvements.

TABLE I

MAIN COMPARISON RESULT. FOR EACH TASK, **NAUC** AND **Final SR** ARE REPORTED IN SEPARATE COLUMNS, IN PERCENT, AVERAGED OVER THREE SEEDS.

Method	Space	Can		Square		Transport		PegInsertionSide		StackCube		Average	
		NAUC	Final SR	NAUC	Final SR	NAUC	Final SR	NAUC	Final SR	NAUC	Final SR	NAUC	Final SR
Base policy	Base	/	87.4	/	57.8	/	54.0	/	49.7	/	40.0	/	57.8
DSRL [4]	Noise	90.6	95.8	55.7	66.1	58.5	75.7	46.4	47.9	38.5	45.4	57.9	66.2
LPS [7]	Noise	91.4	90.0	58.0	69.4	22.5	55.4	9.1	1.4	27.3	34.8	41.7	50.2
LP-DS [8]	Noise	93.0	94.4	57.5	59.4	55.2	55.0	52.5	51.8	41.2	41.9	59.9	60.5
CNDO	Noise	91.6	96.0	<u>68.1</u>	80.6	<u>71.2</u>	87.7	<u>65.8</u>	<u>83.1</u>	50.9	60.1	<u>69.5</u>	81.5
DICE [6]	Action	92.9	97.4	63.8	<u>82.2</u>	68.4	<u>89.8</u>	47.5	72.9	<u>70.0</u>	94.1	68.5	<u>87.3</u>
DSRL+DICE	Joint	90.7	97.1	66.0	78.9	67.2	83.4	57.9	70.3	58.2	83.3	68.0	82.6
CANDO	Joint	93.2	<u>97.2</u>	72.2	89.4	74.1	91.7	70.9	88.2	70.4	<u>90.6</u>	76.1	91.4

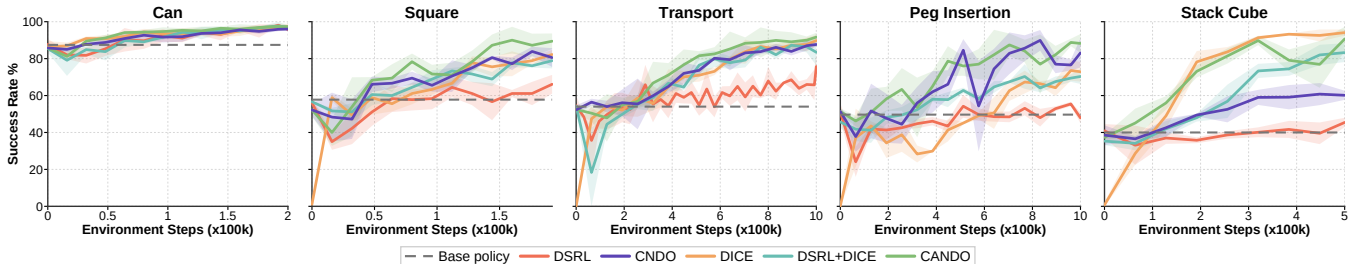


Fig. 6. Joint noise-action residual learning on five tasks. Lines and shaded regions denote the mean and sample standard deviation over three seeds; the gray dashed line is the frozen base policy.

D. Action-Noise Joint Residual Learning

We compare the standalone and joint versions of the noise-space and action-space learners to answer the **Q3**, the results shown in Fig. 6. In particular, we contrast the direct combination DSRL+DICE with CANDO to determine whether improving the standalone noise learner changes the benefit of joint training.

The standalone DSRL learner trails DICE by 10.6 average NAUC points. Directly combining them does not improve the stronger branch: DSRL+DICE obtains 68.0 average NAUC and 82.6% Final SR, compared with 68.5 and 87.3% for DICE. Its task-level gains on Square and PegInsertionSide are offset by losses on Can, Transport, and StackCube.

After improving the standalone noise learner, CNDO and DICE are closely matched in average NAUC. Their joint model CANDO reaches 76.1 average NAUC, gains of 6.6 points over CNDO and 7.6 points over DICE, and raises average Final SR to 91.4%. CANDO achieves either the best or second-best NAUC and Final SR across all five tasks, but it does not consistently outperform the single-space methods on every task. This pattern is particularly clear on PegInsertionSide and StackCube. On PegInsertionSide, CNDO substantially outperforms DICE, while CANDO performs similarly to CNDO. The opposite pattern is observed on StackCube.

These results empirically support the proposed relationship: joint training is most useful here after the two standalone residual learners have comparable capability.

E. Real World Experiments

We validate CANDO and CNDO on and in real-world setting, where a Simplexity Robotics I7 dual-arm robot with two Wuji hands and three cameras.

V. CONCLUSION

This work introduced Conservative Noise Directional Optimization (CNDO) and Conservative Action-Noise Directional Optimization (CANDO) for sample-efficient residual RL. CNDO improves noise-space residual learning without imposing a fixed latent feasible region. It optimizes Gaussian-anchored residual directions with conservative value estimation and uses a target-critic supervised gator to selectively execute trusted decoded edits. Across simulation and real world test, CNDO substantially outperforms existing noise-space methods and achieves overall sample efficiency comparable to the strong action-space baseline DICE. Building on this stronger noise-space learner, CANDO jointly optimizes pre-decoding noise directions and post-decoding action residuals at the complete edited action. CANDO achieves the highest average sample efficiency, reaching 76.1 NAUC and 91.4% Final SR, whereas the direct DSRL+DICE combination does not improve over DICE. These results indicate that access to both adaptation spaces is not sufficient by itself: joint residual training is most beneficial when the standalone noise-space and action-space learners have comparable capability.

REFERENCES

- [1] C. Chi, Z. Xu, S. Feng, E. Cousineau, Y. Du, B. Burchfiel, R. Tedrake, and S. Song, "Diffusion policy: Visuomotor policy learning via action diffusion," *The International Journal of Robotics Research*, vol. 44, no. 10-11, pp. 1684–1704, 2025.

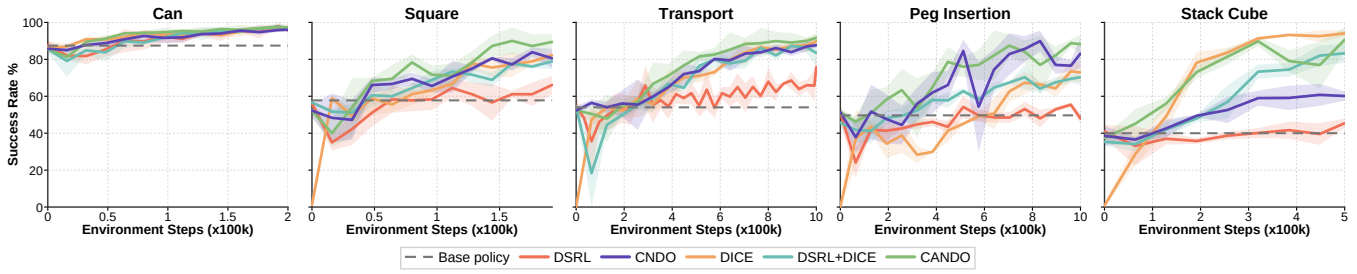


Fig. 7. Real world experiments.

- [2] E. Chisari, N. Heppert, M. Argus, T. Welschehold, T. Brox, and A. Valada, "Learning robotic manipulation policies from point clouds with conditional flow matching," in *Proceedings of the 8th Conference on Robot Learning*, ser. Proceedings of Machine Learning Research, vol. 270. PMLR, 2025, pp. 982–993.
- [3] S. Ross, G. Gordon, and D. Bagnell, "A reduction of imitation learning and structured prediction to no-regret online learning," in *Proceedings of the fourteenth international conference on artificial intelligence and statistics*. JMLR Workshop and Conference Proceedings, 2011, pp. 627–635.
- [4] A. Wagenmaker, Y. Zhang, M. Nakamoto, S. Park, W. Yagoub, A. Nagabandi, A. Gupta, and S. Levine, "Steering your diffusion policy with latent space reinforcement learning," in *Proceedings of the 9th Conference on Robot Learning*, vol. 305. PMLR, 2025, pp. 258–282.
- [5] T. Johannink, S. Bahl, A. Nair, J. Luo, A. Kumar, M. Loskyll, J. A. Ojea, E. Solowjow, and S. Levine, "Residual reinforcement learning for robot control," in *2019 International Conference on Robotics and Automation (ICRA)*. IEEE, 2019, pp. 6023–6029.
- [6] Z. Sun and S. Song, "From prior to pro: Efficient skill mastery via distribution contractive RL finetuning," in *Forty-third International Conference on Machine Learning*, 2026, pp. 1–17.
- [7] H. Im, A. Kolobov, J. Fu, and Y. Lee, "Latent policy steering through one-step flow policies," in *Proceedings of Robotics: Science and Systems*, 2026, pp. 1–14.
- [8] H. Simsir and O. S. Oguz, "Lagrangian perturbation diffusion steering: Latent reinforcement learning for generative policies," *arXiv preprint arXiv:2606.01151*, 2026.
- [9] T. Silver, K. R. Allen, J. B. Tenenbaum, and L. P. Kaelbling, "Residual policy learning," *arXiv preprint arXiv:1812.06298*, 2018.
- [10] G. Schoettler, A. Nair, J. Luo, S. Bahl, J. A. Ojea, E. Solowjow, and S. Levine, "Deep reinforcement learning for industrial insertion tasks with visual inputs and natural rewards," in *2020 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2020, pp. 5548–5555.
- [11] C. Schaff and M. R. Walter, "Residual policy learning for shared autonomy," in *Proceedings of Robotics: Science and Systems*, Jul. 2020, pp. 1–10.
- [12] M. Alakuijala, G. Dulac-Arnold, J. Mairal, J. Ponce, and C. Schmid, "Residual reinforcement learning from demonstrations," *arXiv preprint arXiv:2106.08050*, 2021.
- [13] T. Davchev, K. S. Luck, M. Burke, F. Meier, S. Schaal, and S. Ramamoorthy, "Residual learning from demonstration: Adapting DMPs for contact-rich manipulation," *IEEE Robotics and Automation Letters*, vol. 7, no. 2, pp. 4488–4495, 2022.
- [14] W. Zhou, S. Bajracharya, and D. Held, "PLAS: Latent action space for offline reinforcement learning," in *Proceedings of the 2020 Conference on Robot Learning*, ser. Proceedings of Machine Learning Research, J. Kober, F. Ramos, and C. Tomlin, Eds., vol. 155. PMLR, 2021, pp. 1719–1735.
- [15] A. Ajay, A. Kumar, P. Agrawal, S. Levine, and O. Nachum, "OPAL: Offline primitive discovery for accelerating offline reinforcement learning," in *International Conference on Learning Representations*, 2021, pp. 1–23.
- [16] K. Pertsch, Y. Lee, and J. J. Lim, "Accelerating reinforcement learning with learned skill priors," in *Proceedings of the 2020 Conference on Robot Learning*, ser. Proceedings of Machine Learning Research, J. Kober, F. Ramos, and C. Tomlin, Eds., vol. 155. PMLR, 2021, pp. 188–204.
- [17] X. Chen, A. Ghadirzadeh, T. Yu, J. Wang, A. Y. Gao, W. Li, L. Bin, C. Finn, and C. Zhang, "LAPO: Latent-variable advantage-weighted policy optimization for offline reinforcement learning," in *Advances in Neural Information Processing Systems*, vol. 35, 2022, pp. 36902–36913.
- [18] T. Yoneda, L. Sun, G. Yang, B. C. Stadie, and M. R. Walter, "To the noise and back: Diffusion for shared autonomy," in *Proceedings of Robotics: Science and Systems*, 2023, pp. 1–18.
- [19] L. Ankile, A. Simeonov, I. Shenfeld, M. Torne, and P. Agrawal, "From imitation to refinement: Residual RL for precise assembly," in *2025 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2025, pp. 1–8.
- [20] X. Yuan, T. Mu, S. Tao, Y. Fang, M. Zhang, and H. Su, "Policy decorator: Model-agnostic online refinement for large policy model," in *International Conference on Learning Representations*, 2025, pp. 1–36.
- [21] L. Ankile, Z. Jiang, R. Duan, G. Shi, P. Abbeel, and A. Nagabandi, "Residual off-policy RL for finetuning behavior cloning policies," *arXiv preprint arXiv:2509.19301*, 2025.
- [22] H. Hu, S. Mirchandani, and D. Sadigh, "Imitation bootstrapped reinforcement learning," in *Proceedings of Robotics: Science and Systems*, 2024, pp. 1–18.
- [23] G. Ma, L. Li, H. Wang, Z. Liu, P.-L. Bacon, and D. Tao, "What makes value learning efficient in residual reinforcement learning?" *arXiv preprint arXiv:2602.10539*, 2026.
- [24] P. J. Ball, L. Smith, I. Kostrikov, and S. Levine, "Efficient online reinforcement learning with offline data," in *International Conference on Machine Learning*. PMLR, 2023, pp. 1577–1594.
- [25] A. Mandlkar, D. Xu, J. Wong, S. Nasiriany, C. Wang, R. Kulkarni, L. Fei-Fei, S. Savarese, Y. Zhu, and R. Martín-Martín, "What matters in learning from offline human demonstrations for robot manipulation," in *5th Annual Conference on Robot Learning*. PMLR, 2021, pp. 1–13.
- [26] T. Mu, Z. Ling, F. Xiang, D. C. Yang, X. Li, S. Tao, Z. Huang, Z. Jia, and H. Su, "Maniskill: Generalizable manipulation skill benchmark with large-scale demonstrations," in *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks*, vol. 1, 2021, pp. 1–14.